```python
"""

Handles reading and writing of application data to disk.
  - favorites.json  → saved cities (JSON)
  - weather_log.csv → weather history log (CSV)

"""

import json
import csv
import os
from datetime import date
from typing import Optional

from models import FavoriteCity, WeatherRecord
from exceptions import FileStorageError


# Paths


DATA_DIR = os.path.join(os.path.dirname(__file__), "data")
FAVORITES_FILE = os.path.join(DATA_DIR, "favorites.json")
LOG_FILE = os.path.join(DATA_DIR, "weather_log.csv")

# CSV column headers

_LOG_HEADERS = [
    "city", "country", "date", "temperature_c", "feels_like_c",
    "humidity_pct", "wind_speed_kmh", "description", "is_forecast",
]


def _ensure_data_dir() -> None:
    """Create the data/ directory if it does not exist."""
```

```python
    os.makedirs(DATA_DIR, exist_ok=True)



# Favorites json favoritos

def load_favorites() -> list[FavoriteCity]:
    """
    Load saved cities from favorites.json.

    Returns an empty list when the file does not exist yet.

    Raises:
        FileStorageError: on corrupt JSON or permission errors.
    """
    _ensure_data_dir()
    if not os.path.exists(FAVORITES_FILE):
        return []
    try:
        with open(FAVORITES_FILE, "r", encoding="utf-8") as fh:
            raw: list[dict] = json.load(fh)
        return [FavoriteCity.from_dict(entry) for entry in raw]
    except (json.JSONDecodeError, KeyError) as exc:
        raise FileStorageError(f"Corrupt favorites file: {exc}") from exc
    except OSError as exc:
        raise FileStorageError(f"Cannot read favorites: {exc}") from exc


def save_favorites(cities: list[FavoriteCity]) -> None:
    """
    Persist cities to favorites.json overwrites existing file.

    Raises:
```

```python
        FileStorageError: on write failures.
    """
    _ensure_data_dir()
    try:
        with open(FAVORITES_FILE, "w", encoding="utf-8") as fh:
            json.dump([c.to_dict() for c in cities], fh, indent=2)
    except OSError as exc:
        raise FileStorageError(f"Cannot write favorites: {exc}") from exc


def add_favorite(city: FavoriteCity) -> list[FavoriteCity]:
    """
    Add *city* to favorites if not already present by name  country
    Returns the updated list
    """
    cities = load_favorites()
    already = any(
        c.name.lower() == city.name.lower() and c.country == city.country
        for c in cities
    )
    if not already:
        cities.append(city)
        save_favorites(cities)
    return cities


def remove_favorite(name: str) -> list[FavoriteCity]:
    """Remove a city by name (case-insensitive). Returns updated list."""
    cities = load_favorites()
    updated = [c for c in cities if c.name.lower() != name.lower()]
    save_favorites(updated)
    return updated
```

```python



# Weather log CSV

def append_to_log(record: WeatherRecord) -> None:
    """
    Append a WeatherRecord row to the CSV log file.
    Creates the file with headers if it does not exist.

    Raises:
        FileStorageError: on write failures.
    """
    _ensure_data_dir()
    file_exists = os.path.exists(LOG_FILE)
    try:
        with open(LOG_FILE, "a", newline="", encoding="utf-8") as fh:
            writer = csv.DictWriter(fh, fieldnames=_LOG_HEADERS)
            if not file_exists:
                writer.writeheader()
            writer.writerow({
                "city": record.city,
                "country": record.country,
                "date": record.recorded_date.isoformat(),
                "temperature_c": record.temperature_c,
                "feels_like_c": record.feels_like_c,
                "humidity_pct": record.humidity_pct,
                "wind_speed_kmh": record.wind_speed_kmh,
                "description": record.weather_description,
                "is_forecast": record.is_forecast,
            })
    except OSError as exc:
        raise FileStorageError(f"Cannot write log: {exc}") from exc
```

```python


def load_log() -> list[dict]:
    """
    Return all rows from the CSV log as a list of dicts.
    Returns an empty list when the file does not exist.

    Raises:
        FileStorageError: on read failures.
    """
    _ensure_data_dir()
    if not os.path.exists(LOG_FILE):
        return []
    try:
        with open(LOG_FILE, "r", encoding="utf-8") as fh:
            return list(csv.DictReader(fh))
    except OSError as exc:
        raise FileStorageError(f"Cannot read log: {exc}") from exc
```